Workshop 3

REST API Endpoints Documentation

Students:

Juan David Castaneda Cardenas
Laura Vanesa Alvarez Lafont
Nicolas Andres Bolanos Fernandez
Santiago Andres Fetecua Pulgarin
Sergio Nicolas Correa Escobar

Professor:

Carlos Andres Sierra Virguez

Software Engineering II
School of Engineering
Universidad Nacional de Colombia

Attendance Microservice

POST /attendance/confirm/ – Confirm Attendance

Endpoint to confirm attendance to an event.

Fill the body with a Attendance JSON containing:

```
JSON
{
  "name": "string",
  "email": "user@example.com",
  "contact_number": "string",
  "doc_id": "string",
  "waitlist": bool,
  "event_assistance_id": int
}
```

Remember to give the capacity of the event using the query parameter capacity: /attendance/confirm/?capacity=#

Response

Success code 201 - {“attendance”: new attendance}

PUT /attendance/update / – Update Attendance

Endpoint to update the data of an attendance to an event.

Fill the body with a Attendance JSON containing:

```
JSON
{
  "name": "string",
  "email": "user@example.com",
  "contact_number": "string",
  "doc_id": "string",
  "waitlist": bool,
  "event_assistance_id": int
}
```

Response

Success code 202 - {“attendance”: new attendance}

GET /attendance/event/{event_id} — Get Attendances

Endpoint to retrieve the attendances of an event,

Response

Success Code 200 - {“data” : [event attendances] }

GET /attendance/check/document/{document_id}/event/{event_id} — Check if Confirmed

Check if a user with document (document_id) has confirmed his/her attendance to the event(event_id)

Response

Success Code 200 - {“response” : attendance or False}

PUT /attendance/waitlist/switch/id/{document}/event/{event_id}— Switch Wait List Status

Switch the waitlist status of a user for a specified event.

Response

Success Code 202 - {“attendance” : attendance}

Authentication Microservice

POST /auth/register – Register User

Endpoint to create a new user account. Validate name, username, email and password before creating.
Fill the body with a Attendance JSON containing:

```json
{
  "name": "string",
  "username": "string",
  "email": "user@example.com",
  "password": "string"
}
```

Response
Success code 201. Empty body (created).
Error code 409 - Body: string message (e.g. "Email already used" or validation error).

Validation notes
email: must be a valid email format.
username: alphanumeric / . _ - , 3–30 chars.
password: minimum 8 chars, must include letters and digits (adjust to project rules).

POST /auth/login – Authenticate User

Endpoint to obtain a JWT. Accepts either email or username as the identifier.
Fill the body with a Attendance JSON containing:

```
JSON
{
 "identifier": "user@example.com | username",
  "password": "string"
}
```

Response
Success code 200 - { "token": "jwt-token" }
Error code 409 - Body: string message (e.g. "Invalid credentials").

Validation notes
If the identifier contains '@' it is treated as an email and validated accordingly.

POST /auth/login – Authenticate User

Endpoint to obtain a JWT. Accepts either email or username as the identifier.
Fill the body with a Attendance JSON containing:

```
JSON
{
 "identifier": "user@example.com | username",
  "password": "string"
}
```

Response
Success code 200 - { "token": "jwt-token" }

Error code 409 - Body: string message (e.g. "Invalid credentials").

Validation notes

If the identifier contains '@' it is treated as an email and validated accordingly.

/auth/userinfo – Get User Information

Api-gateway Microservice

The API Gateway is the centralized entry point for all REST requests made by the frontend. Instead of calling each microservice individually, the frontend sends every HTTP request to the API Gateway, which forwards it to the correct backend microservice (Attendance Service, Event Manager Service, Login Service, etc.).

The Gateway ensures:

- Unified access to all microservices
- URL simplification for the frontend
- Separation of concerns
- Centralized routing
- Easier maintenance and evolution of the system

Base URL: http://localhost:8003

Attendance Endpoints (API Gateway)
Confirms attendance to an event.

- **POST /attendance/confirm**
  Request body

```
{
  "name": "string",
  "email": "user@example.com",
  "contact_number": "string",
  "doc_id": "string",
  "waitlist": false,
```

"event_assistance_id": 1
}

Query parameter
capacity=<number>

Response(201)
{ "attendance": new_attendance }

- **PUT /attendance/update**
  Updates an existing attendance entry.

  Response(202)
  { "attendance": updated_attendance }
- **GET /attendance/event/{event_id}**
  Retrieves all attendances for the given event.

  Response(200)
  { "data": [...] }

- **GET /attendance/check/document/{document_id}/event/{event_id}**
  Checks whether a user has already confirmed attendance.

  Response(200)
  { "response": attendance_or_false }

- **PUT /attendance/waitlist/switch/id/{document}/event/{event_id}**
  Switches the waitlist status of an attendee.

  Response(202)
  { "attendance": attendance }

Event Manager Endpoints (API Gateway)

- **POST /events/ – Create Event**
  Creates a new event.

  Body:

  ```
  {
   "name_event": "string",
   "date": "YYYY-MM-DD",
   "time": "HH:MM",
   "location": "string",
   "attendance_capacity": number,
   "creator_id": number
  }
  ```

  Response
  Returns the created event.

- **GET /events/user/{user_id} – Get Events by User**
  Returns all events created by the specified user.

- **DELETE /events/{event_id} – Delete Event**
  Deletes the event with the given ID.

  Response
  Success Code 200
  { "detail": "Event deleted"}

  If the event does not exist:
  Error Code 404
  {"detail": "Event does not exist"}

- **GET /events/{event_id}/share – Generate Shareable Link**
  Generates a temporary share link for the event.

{ "share_link": "https://miapp.com/formulario?event_id=_"}

`event_id`: ID of the event.

Login Endpoints (API Gateway)

These endpoints allow the frontend to authenticate users through the API Gateway.

- **POST /auth/register**

  Registers a new user.

  Body:

  ```
  {
    "name": "string",
    "username": "string",
    "email": "string",
    "password": "string"
  }
  ```

  Response

  201 Created — user registered.

- **POST /auth/login**

  Authenticates a user and returns an access token.

  Body:

  ```
  {
    "identifier": "email or username",
    "password": "string"
  }
  ```

  Response

  {
  "accessToken": "JWT_TOKEN"
  }

- **POST /auth/refresh**

  Requests a new access token.

- **POST /auth/userinfo**

  Retrieves user information from a user ID.

  Body

  ```
  {
    "id": number
  }
  ```

  Response

  ```
  {
    "email": "string",
    "name": "string"
  }
  ```